

R3.08 : CONSOLIDATION DE LA PROGRAMMATION

A.ALBERT

Sommaire

- * Introduction et rappels
- * Notion de classes
- * Format d'écriture et de lecture

Fiche PN



Description

Utiliser la [syntaxe Markdown](#) dans la mesure du possible ce qui nous évitera de reprendre chacune des fiches, merci pour les relecteurs

Contexte et ancrage professionnel :

Cette ressource approfondit les bases de la programmation. Elle permet un traitement automatisé en utilisant des structures de données plus performantes.

Contenus :

L'étudiant abordera les notions suivantes :

- Principes fondamentaux de la programmation orientée objet :
 - Classes/Objets/attributs/méthodes/constructeurs
 - Notion d'héritage, agrégation
 - Format et description de données (affichage, [expr. textuelle](#) pour un [json/yaml/xml](#), date avec [timezone](#), retour sur l'encodage)
- Sérialisation des objets (texte versus binaire)
- Gestion des erreurs/exceptions

Prolongements possibles :

- Les interfaces, classes abstraites, polymorphisme
- Structures de données avancées comme par exemple listes, files, piles, arbres, ...



Mots clés

POO, classes, objet, algorithmes, sérialisation, encodage, exceptions

Utilisez la “,” pour séparer les

Apprentissages critiques visés

- * AC23.02 | développer une application à partir d'un cahier des charges donné, pour le web ou les périphériques mobiles
- * AC23.05 | Accéder à un ensemble de données depuis une application ou un site web

Organisation du module

- * 1 cours magistraux
- * 1 Examen sur table (UML + cours)
- * 4 TD
- * 3 TP
- * **1 TP Test**

Introduction

- * Fait suite aux ressources R1.07 fondamentaux de la programmation et R2.08 analyse et traitement de données structurées
- * Aborde les notions de programmation orientée objet, classes, agrégation, héritage, sérialisation, ...

Rappel

- * Python est un langage de programmation interprété, multi-plateformes au multiples usages
- * Variables non typées (espace des noms et espace des valeurs)
- * Permet de faire de la programmation orienté objet (dans les faits, en python tout est objet)

Annotations de type

- * Python ne type pas les variables
- * Mais on peut indiquer quel type de données est attendu => annotation
- * Exemples:

```
a : int = 3 # la variable a espère un entier
```

```
def function (a : int, b : str ) -> str : # une fonction qui prend un  
entier a, une chaine de caractère b est qui retourne une chaine de  
caractère
```


Méthodes python

- * Les arguments des méthodes peuvent être
 - * Positionnel => à la position qu'ils ont dans la déclaration de la méthode
 - * Nommé => précéder du nom de la variable associée et la il peuvent être dans n'importe quel ordre
 - * Sous forme d'un tuple de valeur non nommées => elle sont utilisées dans l'ordre
 - * Sous la forme d'un dictionnaire => c'est un ensemble de valeurs nommées

```
def fonction (p1 , p2, n1=val1, n2=val2, *args, **kwargs)
```

Arguments positionnels

Arguments nommés (forme nom = Valeur default) Liste d'arguments variables -> tuple

Liste d'arguments nommés
variables -> dictionnaire

Portée des variables

- * Dans le cadre des fonctions et variables classiques, la recherche s'effectue selon la règles LEGB
 - * L : Local
 - * E : Englobant (c'est à dire au niveau de la fonction qui définit l'autre fonction
 - * G : Global
 - * B : éléments built-in. Pour rappel les fonctions et variables built-in sont tous les éléments disponibles par défaut sans aucun autre module. Pour plus d'info, taper `help(builtin)` dans un interpréteur python

Documenter son code

- * Il est important de documenter son code (c'est une des composantes essentielles de la compétence **programmer**)
- * Il existe plusieurs standards de documentation avec leur structure propre
- * Certains outils permettent de lire cette documentation et de la transformer en documentation html, pdf, ...
- * Il est important lorsqu'on génère la documentation de structurer son répertoire de développement

Structuration du répertoire

- * Project root
 - * Project
 - * src
 - * tests
 - * Docs
 - * Readme.txt

Documentation : chaine de documentation google

```
""" documentation de fonction ou methode
fonction permettant de calculer la distance entre deux points d'un plan en 2 dimenations

Args :
    abscisse (float): abscisse du second point
    ordonnee (float): ordonnée du second point

Returns:
    float : distance entre les deux points

    cette distance est calculée à l'aide de la relation de pythagore

Raises :
    AttributeError : renvoie cette exception si les arguments ne sont pas de type réels

"""
```


Documentation reStructuredText

```
""" documentation de fonction ou methode
fonction permettant de calculer la distance entre deux points d'un plan en 2 dimenations

:param abscisse: abscisse du second point
:type abscisse: float
:param ordonnee: ordonnée du second point
:type ordonnee : float

:returns: distance entre les deux points. cette distance est calculée à l'aide de la relation de pythagore
:rtype: float

:raise: AttributeError : renvoie cette exception si les arguments ne sont pas de type réels

"""
```


Docstrings NumPy / SciPy

```
""" documentation de fonction ou methode
fonction permettant de calculer la distance entre deux points d'un plan en 2 dimenations

Parameters
-----

abscisse: float
    abscisse du second point
ordonnee: float
    ordonnée du second point

Returns
-----

float
    distance entre les deux points. cette distance est calculée à l'aide de la relation de pythagore

Raises
-----
AttributeError
    renvoie cette exception si les arguments ne sont pas de type réels

"""
```


PROGRAMMATION ORIENTÉ OBJET

Notions d'objet

- * Le stockage des données peut prendre plusieurs forme en python:
 - * Variable simple (données simples)
 - * Liste, Tuple, Dictionnaire (données multiples)
- * La modélisation objet permet de représenter les données sous forme encapsulées
- * À ces données sont associées des méthodes permettant leur manipulation

Classes (**rappel**)

- * Définie par le mot clé **class** dans un module.
- * Possède son propre espace de nommage
- * Peut implémenter des méthodes spéciales (`__init__`, `__str__`, ...)
- * On crée une instance de la classe en utilisant
 - * Variable = nom de la classe (liste des arguments à passer à la méthode `__init__`)
 - * Exemple : `t = maClasse("bonjour", 12)`
 - * Usage des méthodes `objet.nom_méthode`

Modélisation objet : UML

- * l'UML, Unified Modeling Language, est un langage descriptif à base de pictogramme permettant de représenter des schémas relationnels d'objet, de bases de données, de gestion de processus
- * En POO: un objet est décrit dans un cadre par ses attributs, ses constructeurs et ses méthodes. Les relations d'agrégation et d'héritage sont symbolisés par des liens

UML

- * Une personne est définie par un nom, un prénom et une date de naissance.
- * Elle doit pouvoir se présenter et donner son âge.

Personne

- nom : string
- prenom : string
- date_de_naissance : date

+ Personne(n : string, p : string, dn : date)

+ presentation () : string
+ age() : string

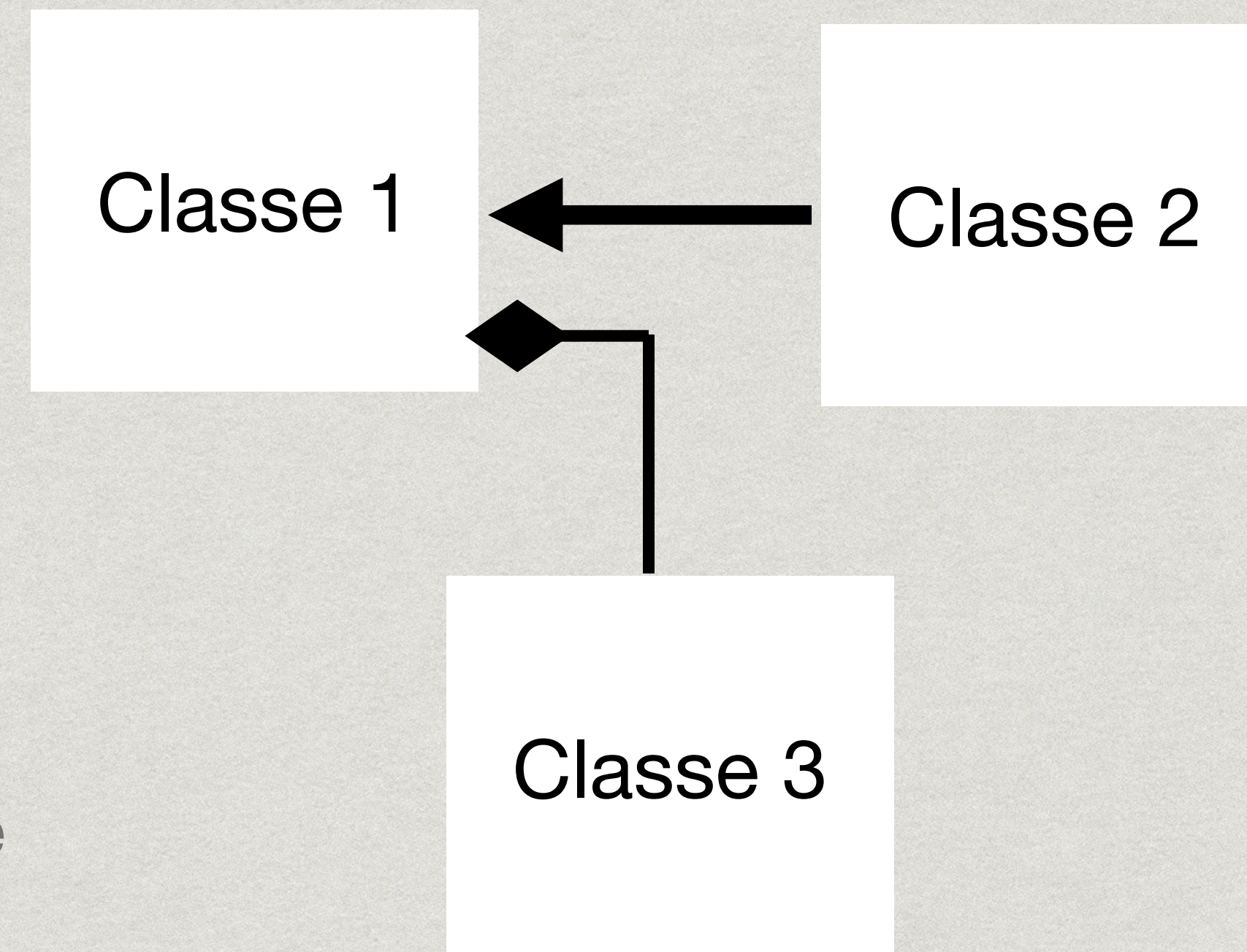
} Attributs

} Constructeur

} Méthodes

UML :

- * Les visibilités sont symbolisées par
 - * + : publique
 - * # : protégé
 - * - privé
- * Les relations sont symbolisées par
 - * Une flèche pour l'héritage => la classe 2 hérite de la classe 1
 - * Un lien terminé par un losange pour une agrégation => la classe 1 possède un attribut du type de la classe 3



Attributs

- * Les attributs sont des données simples encapsulées dans l'objet
- * Ils correspondent aux qualités de l'objet
 - * Ex: un étudiant est décrit par son numéro d'étudiant, son nom et son prénom
- * Ils peuvent être définis au niveau de la classe ou au niveau de l'objet (instance)
- * Un attribut peut être du type d'une autre classe => **AGRÉGATION**

Visibilité des attributs

- * En programmation orienté objet, on définit 3 niveaux de visibilité:
 - * Publique : visible partout
 - * Protégé : visible dans la classe et les classe filles (voir héritage plus tard)
 - * Privé : visible uniquement dans la classe
- * L'intérêt est de protéger l'accès aux attributs de la classe et maîtriser la gestion de ces attributs (écriture)

Visibilité des attributs : Python

- * En python les attributs sont toujours publiques, python n'implémente pas les autres niveaux.
- * Par contre il existe une convention pour émuler les deux autres
 - * `__attribut` : privé
 - * `_attribut` : protégé
- * Malgré tout, on peut continuer à accéder aux attributs avec la notation protégée directement en ajoutant le underscore
- * L'accès direct aux membres notés privés générera une erreur (enfin c'est presque sécurisé) => **on favorisera cette notation afin d'être plus proche de la logique POO**

Constructeur

- * Pour créer une instance de la classe avec des attributs d'instance, il faut ajouter la méthode `__init__`(self, ...)
- * Cette méthode permet d'affecter une valeur à tous les attributs d'instance (ce sont les attributs spécifiques à un objet particulier)

```
Class Personne:  
    pays = "France" # attribut de classe  
  
    def __init__(self, nom, prenom, naissance):  
        # attributs d'instance  
        self.__nom = nom  
        self.__prenom = prenom  
        self.__naissance = naissance
```


Constructeur : valeurs par défaut

- * Comme toutes les fonctions Python, vous pouvez définir des valeurs par défaut dans les arguments du constructeur.

```
Class Personne:
```

```
    Pays = "France" # attribut de classe
```

```
    def __init__(self, nom, prenom = "Matthieu", date_naissance=datetime.now()):
```

```
        # attributs d'instance
```

```
        self.nom = nom
```

```
        self.prenom = prenom
```

```
        self.date_naissance = date_naissance
```


Instanciación d'un objet

- * Variable = nom de classe (valeurs attendues par la méthode init)

```
# Création d'un étudiant avec le constructeur complet
```

```
p1 = Personne("ALBERT", "Arnauld", "01/01/1970")
```

```
print(f"nom : {p1.prenom}") => renvoie Arnauld
```

```
# Création d'un étudiant avec les valeurs par défaut
```

```
p2 = Personne(nom="ALBERT") # usage des arguments nommés
```

```
print(f"nom : {p2.prenom}") => renvoie Mathieu, la valeur par défaut
```


Hack Python: @dataclass

- * Pour éviter de coder de multiple fois les mêmes méthodes : <https://docs.python.org/3/library/dataclasses.html>

- * Implémente de façon automatique `__init__()`, `__repr__()`, `__eq__()`

- * Peut aussi être étendu sur les méthodes de comparaison, définir des valeurs par défaut

- * `@dataclass (frozen = True, eq = False, init = False)`

- * Les attributs peuvent avoir une valeur par défaut au moment de la définition

```
>>> from dataclasses import dataclass
>>> @dataclass
... class Personne:
...     nom:str
...     prenom:str
...
>>> prof = Personne("ALBERT","Arnauld")
>>> prof
Personne(nom='ALBERT', prenom='Arnauld')
>>> prof2 = Personne("ALBERT","Arnauld")
>>> prof2 == prof
True
>>> |
```

- ◆ Frozen : l'objet est non mutable
- ◆ Eq = False : la méthode de test d'égalité n'est pas implémentée
- ◆ Init = False : ne génère pas de `__init__()`, il faut le créer manuellement

Les différentes méthodes

- * Les méthodes sont des fonctions associées à la classe ou à une instance de cette classe
- * Méthode associée à un objet:
 - * La liste des arguments commence toujours par **self**
 - * Elle est appelée au travers d'un objet : **objet.nomMéthode**(valeurs args sans le self)
 - * Elle est liée à l'instance, donc l'objet en lui même est passé comme premier argument de façon automatique
- * Méthode associée à une classe
 - * Elle est précédée du décorateur **@classmethod**
 - * La liste des arguments commence par **cls**
 - * Elle est appelée au travers de la classe : **classe.nomMéthode**(valeurs args sans le cls)
 - * Elle peut accéder aux attributs de classes

Les différentes méthodes

- * Méthodes statiques
 - * Sont précédées du décorateur `@staticmethod`
 - * N'ont pas de premier argument spécifiques contrairement aux deux précédentes elles et sont appelée de la même façon que les méthodes de classes
 - * N'accèdent a aucune variable d'instance ou de classe

Méthodes : exemple

Class Personne:

```
# méthode d'instance  
def presentation(self) -> str :  
    return f"{self.prenom} {self.nom}"
```

```
# méthode de classe, précédé par le décorateur  
@classmethod  
def bonjour2(cls) ->str:  
    return f" bonjour de la classe {cls.__name__}"
```

```
# méthode statique, précédé par le décorateur  
@staticmethod  
def bonjour3() -> str:  
    return "un bonjour statique"
```


Méthodes : usage

- * À partir d'une instance existante :
- * Objet.nom de méthode(valeur d'argument)

```
# instantiation d'une personne
p1 = Personne ("ALBERT","Arnauld","1970-01-01")
#appel de la méthode présentation
salut = p1.présentation()
print(salut) # le résultat affiche "Arnauld ALBERT"
```


Accesseurs

- * Afin de maîtriser l'accès aux attributs, en POO on crée des méthodes pour modifier et récupérer les valeurs d'attributs
- * Getter:
 - * Format def **get***_nom_attribut*() -> type attribut :
- * Setter
 - * Format def **set***_nom_attribut*(args) :
 - * L'argument doit permettre de donner une valeur à l'attribut

Accesseurs en python

- * Afin de maîtriser l'accès aux attributs, en POO on crée des méthodes pour modifier et récupérer les valeurs d'attributs
- * Plusieurs méthodes permettent de définir les accesseurs en Python:
 - * On définit des méthodes get et set pour chaque attribut, qu'on appelle explicitement
 - * On peut aussi les associer aux méthodes spéciales en utilisant la méthode property
 - * On définit des méthodes et on utilise le décorateur @property pour rendre l'accès aux attributs transparent
 - * On redéfinit les méthodes spéciales `__setattr__`, `__getattr__`, et `__delattr__` (avancé)

Getter et setter standard

- * On définit les méthodes set et get pour chaque attribut.

```
class Personne:
    pays = "France" # attribut de classe

    def __init__(self, nom, prenom="Matthieu",
date_naissance=datetime.now()):
    # attributs d'instance
    self.__nom = nom
    self.__prenom = prenom
    self.__date_naissance = date_naissance
    def get_nom(self):
        return self.__nom
    def set_nom(self, nom):
        self.__nom = nom
```


La configuration par la fonction property

- * Cette fonction permet d'affecter les fonctions spéciales permettant de récupérer (GET), modifier (SET), effacer (DEL) pour un attribut.
- * Le prototype de cette fonction est
 - * `property (fget= fonction-get, fset=fonction-set , fdel=fonction-del docs=fonction-doc)`
 - * Les arguments étant nommés, ils sont facultatifs
 - * Ex: `nom = property (get_nom, set_nom)`

Association aux méthodes spéciales

```
class Personne:
    pays = "France" # attribut de classe

    def __init__(self, nom, prenom = "Matthieu", date_naissance=datetime.now()):
        # attributs d'instance
        self.__nom = nom
        self.__prenom = prenom
        self.__date_naissance = date_naissance
    def get_nom(self):
        return self.__nom
    def set_nom(self, nom):
        self.__nom = nom
    nom = property(fget=get_nom, fset = set_nom)
```


Usage de propriétés

- * Nous allons utiliser le décorateur @property pour le getter et @attribut.setter pour la fonction set

```
class Etudiant:
    Promotion = "RT" # attribut de classe

    def __init__(self, nom):
        # attributs d'instance
        self.__nom = nom
    @property
    def nom(self):
        return self.__nom
    @nom.setter
    def nom(self, nom):
        self.__nom = nom
```

À l'usage nous pouvons accéder directement aux attributs, cela appelle les méthodes définies par les propriétés

```
>>> etu = Etudiant("ALBERT")
>>> print(etu.nom)
ALBERT
>>> etu.nom = "DROUHIN"
>>> print(etu.nom)
DROUHIN
```


Accesseur en python (avancé)

- * Lorsque l'on écrit `objet.attribut`, on fait implicitement appel à la méthode `__getattr__(self, name)` ou `name` est le nom de l'attribut.
- * Idem lorsque l'on exécute `objet.attribut = valeur` (méthode `__setattr__(self, name, valeur)`) et `del (objet.attribut)` (méthode `__delattr__(self, name)`)
- * On peut redéfinir explicitement le fonctionnement de ces méthodes en les créant dans le fichier de classe.

Héritage

- * La notion d'héritage est importante en POO
- * Permet de structurer le schéma de données.
- * Exemple :
 - * Nous avons vu la classe personne.
 - * Un étudiant est une personne, mais qui possède aussi une formation et un numéro d'étudiant. Il peut aussi avoir des méthodes spécifiques entre autres sa méthode de présentation qui ne fera pas la même chose que celle de Personne.

Héritage : Python

- * Pour créer une classe héritant d'une autre classe, il suffit de la déclarer entre parenthèse.
- * Une instance d'étudiant peut utiliser les méthodes de la classe Personne
- * La classe Etudiant peut redéfinir une ou plusieurs méthodes de la classe Personne

```
class Etudiant (Personne): # héritage de Personne
    Promotion = "RT" # attribut de classe

    def __init__(self, nom="ALBERT", numero =
12345, groupe ="RT11" ):
    # attributs d'instance
    super().__init__(nom)
    self.numero = numero
    self.groupe = groupe

    def presentation() -> str :
        return f"Étudiant numero {self.numero}
nommé {super.presentation()}"
```


Héritage : mot clé **super**

- * Le mot clé **super** fait référence à la super-classe, la classe dont on hérite (comme dans la méthode présentation)
- * Dans le constructeur, on fait appel à la super classe au travers de `super().__init__(args)` car il est souvent nécessaire (car la méthode `__init__` est surchargée par la classe. On peut aussi utiliser le nom de la classe à la place de `super()`).
- * Le constructeur doit permettre d'initialiser les attributs de la classe dont on hérite et doit donc avoir des valeurs d'arguments pour le constructeur de la super classe.

Portée des variables

- * Dans le cas des variables et fonctions classiques, le mode de recherche est LEGB.
- * Par contre dès qu'on aborde les éléments de classe ou d'instance, le mode de recherche est différent.
- * Instance -> classe -> superclasse et héritage successif. C'est ce qu'on appelle le MRO (Method Resolution Order)

GESTION DES ERREURS

Gestion des erreurs

- * Capture des erreurs lors de l'exécution du code
 - * Evite l'interruption
 - * Permet de traiter l'erreur et d'interagir avec l'utilisateur
- * Génération d'erreur
 - * Permet de spécifier des comportements inappropriés sur l'usage du code produit

Capture des erreurs

- * **try ... except :**
 - * try démarre un bloc de code qui capturera les exceptions possibles
 - * expect : peut être suivi du nom de l'exception et d'un traitement approprié. Il peut y avoir plusieurs clause except
- * **try ... except ... else :** la clause else n'est exécuté que s'il n'y a pas d'erreur capturée. Cela permet d'y mettre le code qui n'est pas problématique
- * **try ... except ... else ... finally :** la clause finally sera exécuté quelle que soit la situation

Gestion des erreurs : Exemple

```
>>> x=10
>>> y=0
>>> try :
...     result = x/y
... except ZeroDivisionError:
...     print("division par zéro interdite")
... else:
...     print(f" résultat {result}")
... finally:
...     print("fin des opérations")
...
division par zéro interdite
fin des opérations
>>> y=2
>>> try :
...     result = x/y
... except ZeroDivisionError:
...     print("division par zéro interdite")
... else:
...     print(f" résultat {result}")
... finally:
...     print("fin des opérations")
...
résultat 5.0
fin des opérations|
>>>
```


Gestion des erreurs

- * Encapsuler uniquement le code susceptible de générer une erreur
- * Distinguer les différentes exécutions possible pour adapter le traitement avec des except différents
- * Toujours garder le except final sans exception pour capturer les autres erreurs possible

Gestion des erreurs : génération d'exception

- * Pour générer une erreur lors de l'exécution d'un code, il faut utiliser l'instruction **raise**.
- * Cette instruction doit être suivie de l'exception levée

```
>>> def fonction(n):
...     if n < 10:
...         raise ValueError("valeur invalide")
...     else :
...         print("valeur supérieur ou égale à 10")
...
...
>>>
>>> n = 9
>>> try :
...     fonction(n)
... except ValueError:
...     print("erreur")
...
... erreur
>>> n = 14
>>> try :
...     fonction(n)
... except ValueError:
...     print("erreur")
...
... valeur supérieur ou égale à 10
>>> |
```


FORMAT DE DONNÉES

JSON , DATE

Format de données : JSON

- * Le JSON est un format concis, lisible très utilisé pour communiquer entre des services tels que les API REST
- * Il est très proche du python avec des dictionnaires et des listes.

```
data = {'nom' : 'ALBERT', 'prenom' : 'Arnauld', 'date_naissance' : '1970-01-01',  
'liste_diplome':[{'nom':'diplome1','annee':1993},{ 'nom':'diplome2','annee':1995}]  
}
```


JSON : utilisation

- * Le format JSON est destiné à être écrit dans un fichier ou un flux pour être transmis.
- * C'est un moyen de sérialisation des données (voir par la suite)
- * Utilise le module **json**
- * Passe d'un dictionnaire à une chaîne de caractère (écriture) ou d'une chaîne de caractère à un dictionnaire (lecture)
- * **json** ne travaille qu'avec des types simples (il n'y a pas d'encodage natif de type complexe ou d'objet)

JSON : exemple

- * *json.loads(x)* : permet de récupérer un dictionnaire issue d'un texte

```
>>> y = json.loads('{"nom":"ALBERT","prenom":"Arnauld","age": 46}')
```

```
>>> y
```

```
{'nom': 'ALBERT', 'prenom': 'Arnauld', 'age': 46}
```

```
>>> y["age"]
```

```
46
```

```
>>> |
```

- * *json.dumps(y)* : permet de transformer un dictionnaire python en chaine de texte json

```
>>> x = json.dumps(y)
```

```
>>> x
```

```
'{"nom": "ALBERT", "prenom": "Arnauld", "age": 46}'
```

```
>>> |
```

- * Les versions load et dump prennent en premier argument un flux contenant ou accueillant le texte json

Format date

- * Le temps est calculé en nombre de seconde écoulé depuis the epoch (1er janvier 1970 00:00:00.00)

```
>>> from time import time
>>> time()
1661773050.739398
>>>
```

Il s'est écoulé 1661773050 secondes
depuis the Epoch

- * Usage du module datetime qui contient la classe datetime
 - * 3 formats disponibles : date (gère les dates), time (gère les heures) et datetime (date et heure)

```
>>> from datetime import date, time, datetime
>>> date(year=2022, month=9, day=2)
datetime.date(2022, 9, 2)
>>> time(12,10,00)
datetime.time(12, 10)
>>> time(12,10,23)
datetime.time(12, 10, 23)
>>> datetime(2023,6,30,18,00,12)
datetime.datetime(2023, 6, 30, 18, 0, 12)
```

Les fonctions peuvent utiliser les arguments nommés
Sinon, date (year, month, day)
time (hour, minutes, secondes, *microsecondes*)
datetime(year, month, day, hour, minutes, secondes, *microsecondes*)

Date et texte

- * Pour importer une date à partir d'une chaîne de caractère:
 - * Méthode `fromisoformat()` de la classe `date` => nécessite d'écrire la date au format anglais `yyyy-mm-dd`
 - * Méthode `strptime` de la classe `datetime` => on peut alors préciser comme premier argument la chaîne de caractères et en second le format de date avec des tags (`%d` pour jour, ...) pour créer le format

Liste des tags : <https://strptime.org>

- * Pour exporter une date en format texte, c'est la méthode `strftime` avec les mêmes tags

Date et texte (2)

- * Plus avancé, nous avons la classe parser du module dateutil (reconnait beaucoup de texte mais en anglais uniquement)

```
>>> from dateutil import parser
>>> parser.parse("2 september 2022 at 9am")
datetime.datetime(2022, 9, 2, 9, 0)
>>>
```

- * Le must, le module dateparser avec sa méthode parse qui interprète beaucoup de textes différents et dans plusieurs langues

```
>>> from dateparser import parse
>>> parse("aujourd'hui")
datetime.datetime(2022, 8, 31, 9, 34, 53, 718484)
>>> parse("2 septembre 2022")
datetime.datetime(2022, 9, 2, 0, 0)
>>> |
```


SÉRIALISATION

Sérialisation des données

- * La sérialisation consiste en des opérations de sauvegarde dans un fichier dans un format qui permette des les extraire simplement pour les récupérer.
- * Il est intéressant de pouvoir l'appliquer sur les objets
 - * Usage de JSON, mais type simple, il faut donc produire des méthodes pour décrire un objet en JSON
 - * Sérialisation binaire en utilisant le module pickle

Sérialisation JSON

- * Pour des objets, classes, nécessite de créer une fonction permettant de mettre en format JSON les données et inversement, de pouvoir créer une classe à travers un dictionnaire issue d'une chaîne JSON
- * Usage des méthodes loads et dumps pour spécifier le fichier d'écriture/lecture.

Sérialisation binaire

- * Le module pickle présente 2 classes, pickler et unpickler et des méthodes pour sérialiser dans un fichier ou sous forme binaire directe.
- * Permet d'utiliser des Pid (persistent id, identifiant unique des objets sérialisés) => cela se déroule avec un héritage de la classe pickler par la classe à sérialiser

Pickle : exemples

- * S rialiser un objet :

```
>>> pickle.dump(file, data) # sauvegarde data sous forme s rialis e dans le fichier
```

```
>>> ser = pickle.dumps(data) # fournit data sous forme s rialis e comme r sultat de l'appel
```

- * D s rialiser un objet :

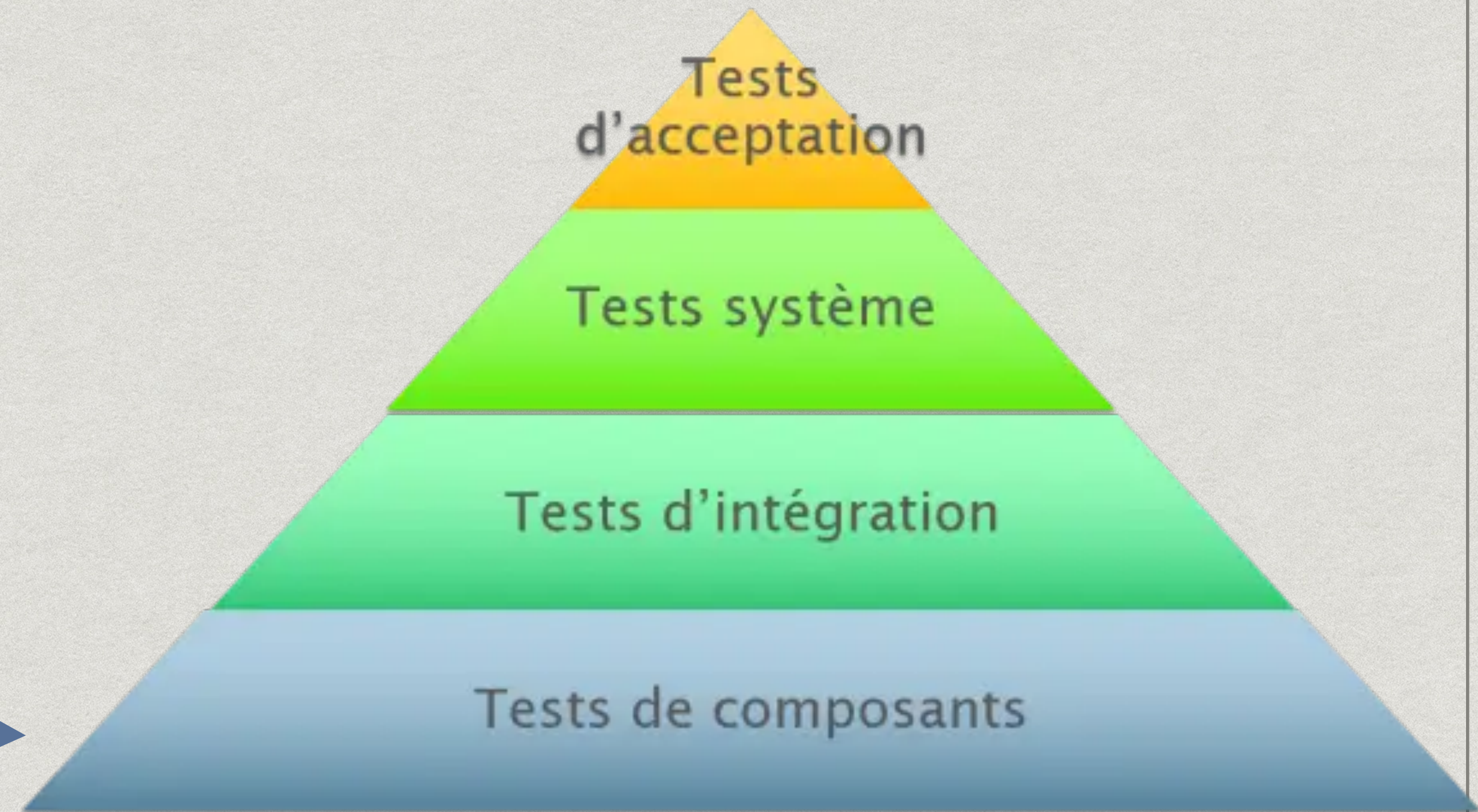
```
>>> data = pickle.load(file) # r cup re data de sa forme s rialis e dans le fichier file
```

```
>>> data = pickle.load(ser) # r cup re data de sa forme s rialis e du binaire ser
```


LES TESTS UNITAIRES

Des test, pourquoi faire ?

- * Avant la fourniture d'un logiciel à l'utilisateur final, il est obligatoire qu'il passe un ensemble de tests
- * La première brique de test, sont les tests de chaque fonctionnalité développée



Les tests unitaires

Framework de tests unitaires

- * En python il existe plusieurs framework pour réaliser les tests unitaires
 - * unittest : framework directement intégré à la librairie standard python
 - * Pytest : un autre framework très utilisé
 - * Doctest : intègre les tests unitaires dans la documentation
 - * Testify, hypothesis, nose2, ...

Framework de tests unitaires

- * En python il existe plusieurs framework pour réaliser les tests unitaires
 - * **Unittest** : framework directement intégré à la librairie standard python
 - * **Pytest** : un autre framework très utilisé
 - * Doctest : intègre les tests unitaires dans la documentation
 - * Testify, hypothesis, nose2, ...

Unittest : intégré directement à Python

- * Nécessite de créer une classe héritant de `unittest.TestCase`
- * Chaque méthode de la classe exécute un ou plusieurs tests sur une fonctionnalité
- * Les tests sont de la forme
 - * `Self.assertEqual(fonction à tester, résultat attendu)`
 - * `Self.assertRaises(exception, fonction à tester)`
 - * ...
- * Les tests sont exécuté par l'appel de la méthode `main` de `unittest`
 - * `unittest.main(argv=[''], verbosity=0, exit=False)`
- * Exécution dans le terminal ou en mode script
 - * `python -m unittest nom_fichier_test.py`

Verbose levels :

- 0 : juste le résultat
- 1 : quelques infos, messages d'erreur générés
- 2 : résultats séparés pour chaque tests

Unittest:

- ✱ La documentation python : <https://docs.python.org/3/library/unittest.html>
- ✱ La classe TestCase et ses méthodes <https://docs.python.org/3/library/unittest.html#unittest.TestCase>

Method	Checks that	New in
assertRaises(exc, fun, *args, **kws)	<code>fun(*args, **kws)</code> raises <code>exc</code>	
assertRaisesRegex(exc, r, fun, *args, **kws)	<code>fun(*args, **kws)</code> raises <code>exc</code> and the message matches regex <code>r</code>	3.1
assertWarns(warn, fun, *args, **kws)	<code>fun(*args, **kws)</code> raises <code>warn</code>	3.2
assertWarnsRegex(warn, r, fun, *args, **kws)	<code>fun(*args, **kws)</code> raises <code>warn</code> and the message matches regex <code>r</code>	3.2
assertLogs(logger, level)	The <code>with</code> block logs on <code>logger</code> with minimum <code>level</code>	3.4
assertNoLogs(logger, level)	The <code>with</code> block does not log on <code>logger</code> with minimum <code>level</code>	3.10

Method	Checks that	New in
assertEqual(a, b)	<code>a == b</code>	
assertNotEqual(a, b)	<code>a != b</code>	
assertTrue(x)	<code>bool(x)</code> is <code>True</code>	
assertFalse(x)	<code>bool(x)</code> is <code>False</code>	
assertIs(a, b)	<code>a is b</code>	3.1
assertIsNot(a, b)	<code>a is not b</code>	3.1
assertIsNone(x)	<code>x is None</code>	3.1

Method	Checks that	New in	
assertAlmostEqual(a, b)	<code>round(a-b, 7) == 0</code>		3.1
assertNotAlmostEqual(a, b)	<code>round(a-b, 7) != 0</code>		3.1
assertGreater(a, b)	<code>a > b</code>	3.1	3.2
assertGreaterEqual(a, b)	<code>a >= b</code>	3.1	b) 3.2
assertLess(a, b)	<code>a < b</code>	3.1	
assertLessEqual(a, b)	<code>a <= b</code>	3.1	3.14
assertRegex(s, r)	<code>r.search(s)</code>	3.1	b) 3.14
assertNotRegex(s, r)	<code>not r.search(s)</code>	3.2	
assertCountEqual(a, b)	<code>a</code> contains the same elements as <code>b</code> , regardless of their order.	3.2	
assertStartsWith(a, b)	<code>a.startswith(b)</code>	3.14	
assertNotStartsWith(a, b)	<code>not a.startswith(b)</code>	3.14	
assertEndsWith(a, b)	<code>a.endswith(b)</code>	3.14	
assertNotEndsWith(a, b)	<code>not a.endswith(b)</code>	3.14	
assertHasAttr(a, b)	<code>hasattr(a, b)</code>	3.14	
assertNotHasAttr(a, b)	<code>not hasattr(a, b)</code>	3.14	

Pytest : environnement de test

- * Module à importer : `pip install -U pytest`
- * Un fichier de test par module à tester (module **personne.py**, fichier **test_personne.py**)
- * Import du module à tester et ensemble de fonctions de test
- * Les fonctions de test sont toutes préfixées par "test_" sinon elles seront ignorées
- * Il faut un test unitaire pour chaque résultat possible des fonctions à tester
- * Utilisation de l'instruction **assert**
 - * `assert fonction(arg1,...) is True`
 - * `assert fonction2(arg1,...) == 12`
- * Test des exceptions avec l'instruction **pytest.raises**, dans un contexte (**with**) englobant la fonction avec des argument générant l'exception

<https://docs.pytest.org/en/stable/index.html>

Pytest : exemple

```
from dataclasses import dataclass
import math
```

```
@dataclass(frozen=True)
class Point:
    x: float
    y: float
```

```
def distance_to(self, other: "Point") -> float:
    """Distance euclidienne vers un autre point."""
    return math.hypot(self.x - other.x, self.y - other.y)

def __add__(self, other: "Point") -> "Point":
    return Point(self.x + other.x, self.y + other.y)
```

```
import pytest
from point import Point
```

```
# ----- Création et attributs de base -----
```

```
def test_creation_valeurs_positives():
    p = Point(3, 4)
    assert p.x == 3
    assert p.y == 4
```

```
def test_creation_valeurs_negatives():
    p = Point(-2, -7)
    assert p.x == -2
    assert p.y == -7
```

```
# ----- Distance -----
```

```
def test_distance_horizontale():
    assert Point(0, 0).distance_to(Point(3, 0)) == pytest.approx(3.0)
```

```
def test_distance_verticale():
    assert Point(0, 0).distance_to(Point(0, 4)) == pytest.approx(4.0)
```

```
def test_distance_triangle_3_4_5():
    assert Point(0, 0).distance_to(Point(3, 4)) == pytest.approx(5.0)
```

```
def test_distance_point_vers_lui_meme():
    p = Point(7, -2)
    assert p.distance_to(p) == pytest.approx(0.0)
```

```
def test_distance_symetrique():
    a, b = Point(1, 1), Point(4, 5)
    assert a.distance_to(b) == pytest.approx(b.distance_to(a))
```

```
def test_distance_avec_coordonnees_negatives():
    a, b = Point(-1, -1), Point(2, 3)
    assert a.distance_to(b) == pytest.approx(5.0)
```


PYTHON ET BASE DE DONNÉES

Accès Base de données

- * Pour connecter à une base de données en python, il faut installer les drivers correspondant
- * Pour mysql, un des modules à utiliser est mysql-connector-python (attention, si vous utilisez mysql-connector, vous risquez d'avoir des problème d'authentification avec les nouvelle versions de mysql)

```
pip install mysql-connector-python
```

- * Comme cette commande accède à des répertoires système, il est préférable de l'installer dans un environnement virtuel python

Mysql et python

- * Pour exécuter des commandes dans le serveur mysql il faut
 1. Établir une connexion avec le serveur
 2. Créer un curseur d'exécution
 3. Exécuter une commande
 4. Si nécessaire, en récupérer le resultset
 5. Le parcourir pour en extraire les données

Mysql et python

- * Établir une connexion:

```
>>> import mysql.connector # attention au point à la place du - du module  
>>> db = mysql.connector.connect(user="root",passwd="motdepasse",host="localhost"  
,database='madatabase')
```

- * Créer un curseur

```
>>> mon_curseur = db.cursor()
```

- * Exécuter une commande

```
>>> mon_curseur.execute("create DATABASE monprojet;")
```

Attention a ajouter un ; à la fin de chaque commande à exécuter. La commande est passée sous la forme d'une chaine de caractères

Mysql et python

- * Commandes d'insertion :

- * Une ligne

```
>>> mon_curseur.execute("insert into eleve(nom, prenom) values ('ALBERT','Arnauld');")
```

```
>>> query = "insert into eleve(nom, prenom) values (%s,%s);"
>>> val = ("ALBERT","Arnauld") # valeur sous forme d'un tuple
>>> mon_curseur.execute(query,val)
```

- * Plusieurs lignes

```
>>> query = "insert into eleve(nom, prenom) values (%s,%s);"
>>> val = [("ALBERT","Arnauld"),("Garinet","Jacques"),("Drouhin","Frédéric")] # valeurs sous
forme d'une liste de tuples
>>> mon_curseur.executemany(query,val)
```

Les requêtes d'insertion étant traitée sous la forme de transaction, elle nécessite un commit

```
>>> db.commit() # réalise dans une transaction l'ensemble des commandes
```


Mysql et python

- * Commandes de lecture:

- * Exécution de la commande

```
>>> mon curseur.execute("select * from eleve;")
```

- * Récupération de toute les entrées

```
>>> res = mon curseur.fetchall()
>>> for row in res:
    print row # affiche le tuple de chaque entrée
```

- * Récupération de la première entrée

```
>>> res = mon curseur.fetchone()
>>> print(res) # affiche le tuple de l'entrée
```